

Exercice 1 : Génération d'un flux de données

L'objectif est de générer un flux de 50 lignes aux caractéristiques suivantes :

- Id : séquence, départ : 1, pas : 1
- Matricule : chaîne aléatoire de 6 caractères
- Age : aléatoire entre 18 et 80
- Date : aléatoire entre « 2007-01-01 » et « 2008-12-31 »
- Pays : France, Chine ou USA

Créer un job que vous appelez Exercice_1.

Récupérer dans la palette le composant tRowGenerator et faites-le glisser dans la zone de développement. Double cliquez sur le composant pour le configurer.

Id : cliquez sur le « + » pour ajouter une colonne dans le flux. Renommez la colonne « Id », changez le type en « Intégrer » et choisissez la fonction « Numeric.sequence ». Le paramétrage de base fait que la séquence va commencer à 1 et aura un pas de 1.

Matricule : cliquez sur le « + » pour ajouter une colonne dans le flux. Renommez la colonne « Matricule ». Le paramétrage de base fait que le type est déjà en « String » et la fonction « TalendString.getAsciiRandomString » est paramétrée pour générer une chaîne de maximum six caractères.

Age : cliquez sur le « + » pour ajouter une colonne dans le flux. Renommez la colonne « Age », changez le type en « Intégrer » et choisissez la fonction « Numeric.random ». Paramétrez la « min value » à 18 et la « max value » à 80.

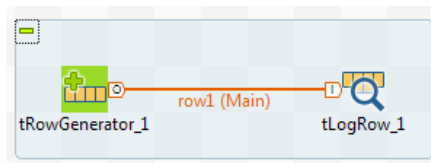
Date : cliquez sur le « + » pour ajouter une colonne dans le flux. Renommez la colonne « Date », changez le type en « Date » et choisissez la fonction « TalendDate.getRandomDate ». Paramétrez la « min value » à « 2007-01-01 » et la « max value » à « 2008-12-31 ».

Pays : cliquez sur le « + » pour ajouter une colonne dans le flux. Renommez la colonne « Pays », changez le type en « String » et choisissez la fonction « ... ». Paramétrez le paramètre personnalisé à "France","Chine","USA".

Renseignez 50 dans le nombre de lignes pour RowGenerator et Validez en cliquant sur « OK ».

Récupérer dans la palette le composant tLogRow, faites le glisser dans la zone de développement et reliez les deux composants en utilisant le clique droit pour générer un row entre les deux. Double cliquez sur le composant tLogRow afin de choisir le mode d'affichage Tableau.

Cliquez sur l'onglet puis le bouton « Exécuter » pour lancer le job et visualiser les résultats dans la console.



Job attendu

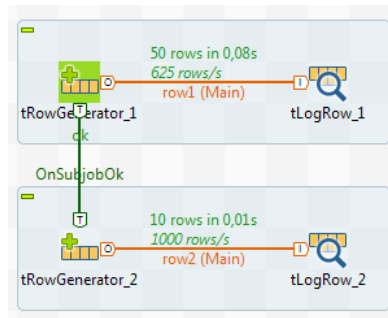
Exercice 2 : Exécution d'un flux de 10 lignes supplémentaires sur réussite de l'exécution du premier flux de 50 lignes

Dupliquez le job Exercice_1 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_2.

Ouvrez le job Exercice_2, sélectionnez les deux composants et faites un copier/coller. Placez le nouveau « sous-job » sous le premier. Modifiez la configuration du second sous-job afin que celui-ci ne génère qu'un flux de 10 lignes.

Faites un clic droit sur le composant tRowGenerator du premier sous-job et sélectionnez « Trigger/OnSubjobOk » que vous reliez au tRowGenerator du second sous-job.

Exécutez le job et vérifiez que les deux flux ont bien été générés à la suite dans la console.



Job attendu

Exercice 3 : Exécution conditionnelle du second flux en fonction du volume de lignes généré par le premier

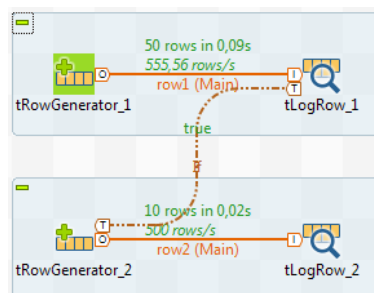
Dupliquez le job Exercice_2 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_3.

Supprimez le lien OnSubjobOk. Créez un lien « Run If » entre le tLogRow du premier flux et le tRowGenerator du second flux. Cliquez ensuite sur le lien If puis sur l'onglet Composant afin de le configurer.

Dans la zone de résumé de contenu « Outline », reperez la variable « Nombre de lignes – NB-LINE (AFTER) » et faites la glisser dans la zone d'édition de la condition. Vous devez alors voir dans la zone condition ((Integer)globalMap.get("tLogRow_1_NB_LINE")). Afin de rendre la condition passante dans un premier temps on va conditionner l'exécution au fait que le tLogRow du premier flux voit 50 lignes. Pour cela ajouter « ==50 » à la condition pour avoir ((Integer)globalMap.get("tLogRow_1_NB_LINE")) == 50.

Exécutez le job et vérifiez que le second flux s'est bien exécuté. Vous pouvez avoir un « true » apparaître au niveau du flux If ce qui signifie que la condition a bien été validée.

Modifiez la condition à 100 lignes et exécutez. Vérifiez que cette fois-ci le second flux n'a pas été exécuté et que le message « false » apparaît sur le flux If.



Job attendu

Exercice 4 : Utilisation de tJava pour afficher un message

L'objectif est d'afficher le message « Début du traitement le 'Date Heure d'exécution'. » à l'exécution du job.

Créer un job que vous appelez Exercice_4.

Récupérer dans la palette le composant tJava et faites le glisser dans la zone de développement. Double cliquez sur le composant pour le configurer.

Dans la zone « Code », allez à la ligne après « String foo = "bar"; » et tapez « syso ». Faites ensuite CTRL+Espace pour utiliser l'autocomplétion et faire apparaître la fonction « System.out.println(); ». Entre les parenthèses, ajoutez « "Début d'exécution du job à " ». Exécutez le job et vérifiez que le message apparait bien dans la console.

Il faut maintenant ajouter la date et heure d'exécution en récupérant la date système. Pour cela, commencez par ajouter « + » après le message dans la parenthèse afin de permettre la concaténation de deux champs. Placez vous ensuite après le « + » et faites CTRL+Espace pour utiliser l'autocomplétion et sélectionnez la fonction « TalendDate.getDate ».

Vous devez alors avoir dans le code :

```
Code      String foo = "bar";
          System.out.println("Début d'exécution du job le
          "+TalendDate.getDate("CCYY-MM-DD hh:mm:ss"));
```

Code

Exécutez le job et vérifiez le message dans la console.

Modifiez le code pour faire apparaître le message sous la même forme que la capture ci-dessous :

```
Démarrage du job Exercice_4 à 17:00
05/09/2018.
[statistics] connecting to socket on port
3864
[statistics] connected
Début d'exécution du job le 05/09/2018 à
17:00:47
[statistics] disconnected
Job Exercice_4 terminé à 17:00 05/09/2018.
[Code sortie=0]
```

Résultat attendu dans la console

Exercice 5 : Ecrire dans un fichier csv

Dupliquez le job Exercice_1 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_5.

Supprimez le tLogRow et insérez un tFileOutputDelimited.

Sur le bureau, créez un répertoire « OUT » dans votre dossier.

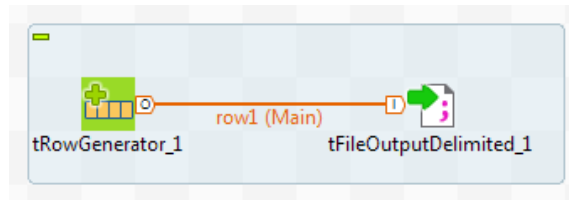
Cliquez sur l'onglet « Composant » après avoir sélectionner le composant tFileOutputDelimited, puis cliquez sur « ... » pour définir l'emplacement et le nom du fichier à créer. Placez-vous dans le dossier que vous venez de créer et tapez « exercice5.csv » dans le nom du fichier.

Exécutez le job et vérifiez que le fichier a bien été créé et qu'il est correct.

Modifiez les paramètres pour :

- afficher les en-têtes de colonnes,
- avoir comme séparateur de champs « @;@ »
- et faire en sorte que les données s'empilent bien dans le fichier.

Supprimez le fichier précédent et exécutez plusieurs fois le job pour vérifier le bon fonctionnement.



Job attendu

Exercice 6 : Dupliquer un flux de données

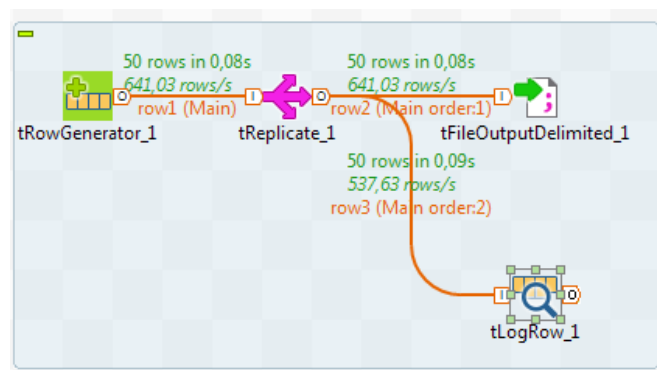
Dupliquez le job Exercice_5 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_6.

Modifiez dans un premier temps le composant tFileOutputDelimited pour que celui-ci écrive dans le fichier exercice6.csv de votre répertoire « OUT ».

Supprimez le flux de données entre les composants et insérez un composant tReplicate. Reliez le tRowGenerator au tReplicate puis le tReplicate au tFileOutputDelimited.

Insérez un tLogRow et placez le sous le tFileOutputDelimited. Reliez le tReplicate au tLogRow.

Exécutez le job et vérifiez le bon fonctionnement.



Job attendu

Exercice 7 : Lire un fichier

Créez dans votre dossier un répertoire « IN » et placez-y le fichier ventes.csv.

Créez un job Exercice_7.

Insérez un composant tFileInputDelimited et cliquez sur « ... » pour que celui-ci lise le fichier ventes.csv de votre répertoire.

Tapez 1 dans la zone « En-Tête » afin d'ignorer la première ligne et cliquez sur « Editer le schéma » afin de le paramétrer comme l'image ci-dessous.

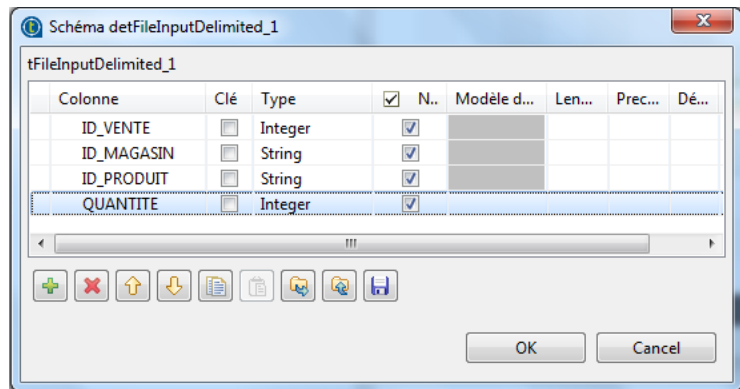
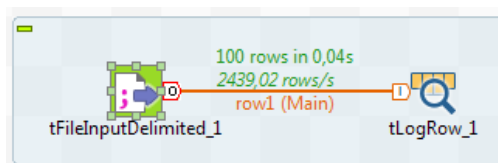


Schéma fichier des ventes

Ajoutez un composant tLogRow que vous reliez au tFileInputDelimited puis exécutez le job.



Job attendu

Exercice 8 : Contrôler la présence d'un fichier avant lecture

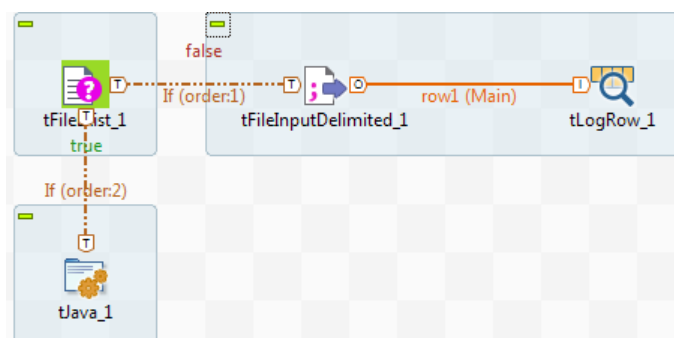
Dupliquez le job Exercice_7 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_8.

Ajoutez un composant tFileExist et cliquez sur « ... » pour qu'il pointe sur le fichier ventess.csv de votre répertoire.

Reliez le tFileExist au tFileInputDelimited avec un lien If puis paramétrez celui-ci en allant récupérer la variable tFileExists.EXISTS (CTRL + Espace) et en faisant le test .equals(true). Ce lien sera donc actif uniquement si le fichier est présent.

Ajoutez un composant tJava pour afficher un message en cas d'absence du fichier. Reliez le tFileExist et le tJava avec un lien if et paramétrez le pour qu'il affiche dans la console le message suivant : Le fichier ventes.csv est absent. Le nom du fichier ne doit pas être écrit en dur mais récupéré à l'aide d'une variable.

Exécutez deux fois le traitement, une fois avec le fichier et une fois sans pour valider votre développement.



Job attendu

Exercice 9 : Filtrer un flux de données

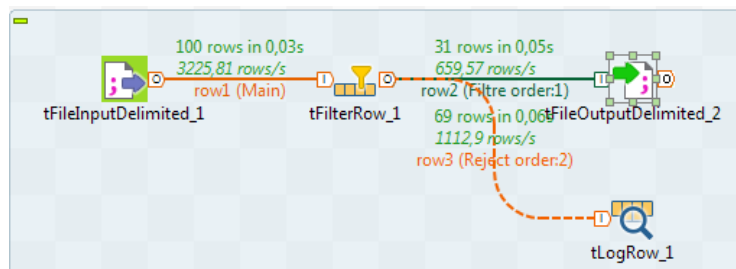
Dupliquez le job Exercice_7 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_9.

Supprimez le composant tLogRow et ajoutez un composant tFilterRow. Nous allons le paramétrer pour envoyer dans un fichier uniquement les lignes correspondantes au Magasin 1 et les autres dans la console. Dans l'onglet composant sélectionnez ID_MAGASIN dans la colonne d'entrée, laissez la fonction à « Vide », sélectionnez l'opérateur « Vaut » et tapez « MAG1 » dans Valeur.

Insérez un composant tFileOutputDelimited et reliez-le au tFilterRow avec le lien « Filtre ». Ecrivez le fichier dans votre répertoire « OUT » et nommez le fichier ventes_magasin1.csv.

Ajoutez un composant tLogRow et reliez-le au tFilterRow avec le lien « Rejet ».

Exécutez le job et vérifiez le bon fonctionnement en contrôlant les lignes dans le fichier et dans la console. Vous devez avoir 31 lignes dans le fichier et 69 dans la console.



Job attendu

Exercice 10 : Agréger et trier des données

Dupliquez le job Exercice_7 en faisant un clic droit sur le job et en sélectionnant « Dupliquer » et nommez le job Exercice_10.

Supprimez le composant tLogRow et ajoutez un composant tAggregateRow. Nous allons le paramétrer pour faire la somme des ventes par magasin. Il faut dans un premier temps définir le schéma de sortie en cliquant sur « Editer le schéma ». Faites passer uniquement dans le schéma de sortie les colonnes ID_MAGASIN et QUANTITE puis validez.

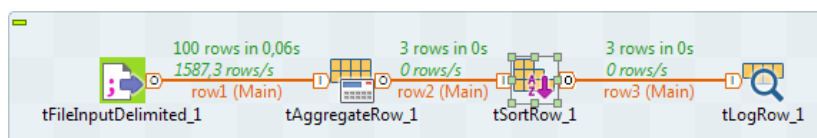
Dans la partie « Group By », ajoutez la colonne ID_MAGASIN qui sera notre colonne d'agrégation.

Dans la partie « Opérations », ajoutez la colonne QUANTITE et sélectionnez la fonction « somme ».

Ajoutez un composant tLogRow afin de rediriger le flux de sortie dans la console et exécutez le job. Vérifiez la bonne exécution du job. On peut voir que les résultats ne sont pas triés.

Ajoutez un composant tSortRow que vous placez entre le tAggregateRow et le tLogRow après avoir supprimé le lien.

Envoyez le flux du tAggregateRow dans le tSortRow puis cliquez sur composant pour le paramétrer. Choisissez la colonne ID_MAGASIN dans « Colonne du schéma » et alpha dans « tri num ou alpha ». Envoyez le flux dans le tLogRow et exécutez le job. Vérifiez que les résultats sont bien triés.



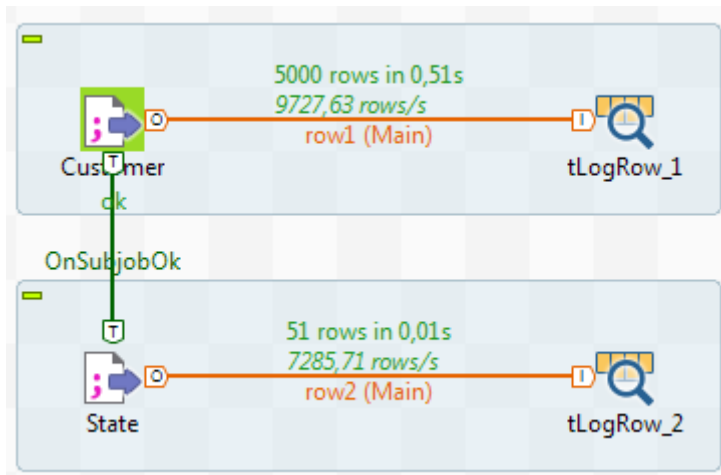
Job attendu

Exercice 11 : Création de métadonnées

Créez un job Exercice_11.

Créez les métadonnées des fichiers Clients et Départements

Dans le job, lisez et envoyez le contenu du fichier Clients dans la console puis sur réussite du sous job faites la même chose avec le fichier Départements.



Job attendu

Exercice 12 : Transformation avec tMap

Etape 1 : Concaténation dans le tMap

Récupérez les données du fichier clients.csv (tFileInputDelimited) et concaténez le nom et le prénom du client dans un nouveau champs nomCompleterGerant. Pour cela, glissez déposez le champs prenomGerant sur le champ nomGerant en sortie. Utilisez l'opérateur + pour concaténer les deux variables et faire en sorte de mettre un espace entre le nom et le prénom.

Exécutez le job et vérifiez les résultats dans la console.

Etape 2 : Transformation d'une donnée

Transformez la valeur du champs nomSociété en passant le contenu de la chaine de caractères en majuscules.

Exécutez le job et vérifiez les résultats dans la console.

Exercice 13 : Jointures avec tMap

Réalisez un job qui a partir des fichiers clients et départements qui fournira en sortie dans la console les éléments suivants :

- idClient (clients),
- nomSociété (clients),
- codeDepartement (Département)
- TypeClient (clients)

- nomCompletGerant (clients)

Exercice 14 : Filtre et rejets avec tMap

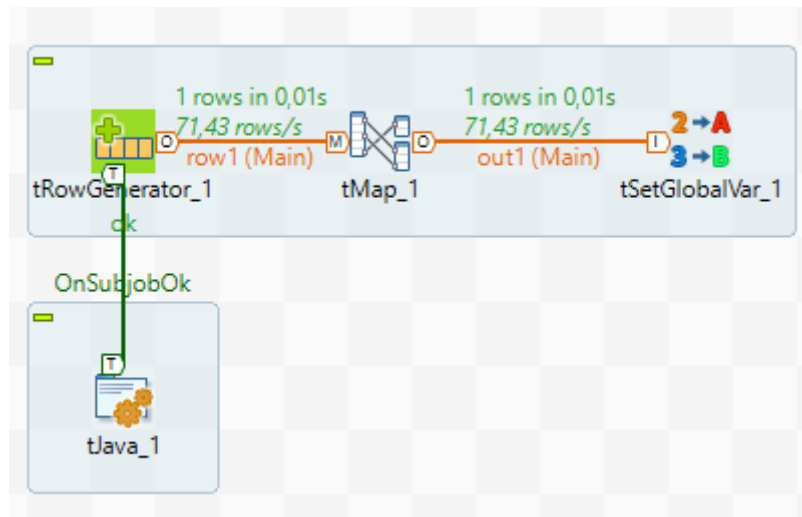
Dupliquez le job précédent, écrire dans des fichiers différents :

- les clients provenant des Pyrénées-Atlantiques dans un fichier client64.csv
- les clients provenant de Gironde dans un fichier clients33.csv
- les autres clients autresclienst.csv

Exercice 15 : Utilisation du tSetGlobalVar

A l'aide d'un tRowGenerator générer la date du jour puis utiliser un tMap pour calculer la date de la veille. Il faut penser à convertir cette date en string car le composant tSetGlobalVar n'accepte en entrée que des String. Fonctions à utiliser : `TalendDate.addDate` et `TalendDate.formatDate`.

Charger cette date calculée dans une variable « dateVeille » et l'afficher dans la console à l'aide d'un tJava.



Job attendu

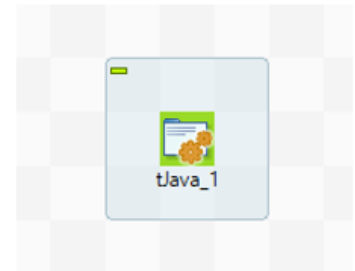
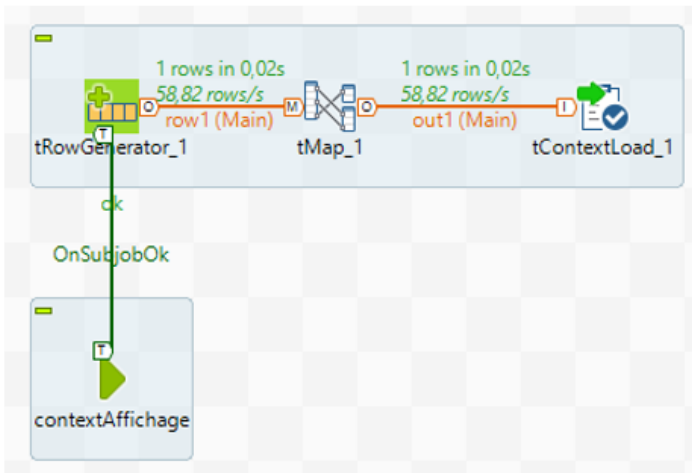
Exercice 16 : Utilisation du tContextLoad

Dans un premier job contextLanceur, cliquer sur l'onglet Contexts et créer une variable dateVeille de type String. Générer de nouveau la date du jour et utiliser un tMap pour la transformer en date de la veille. Vu que nous utilisons un tContextLoad les colonnes de sorties du tMap doivent se nommer key et value. Dans value mettre en dur le nom de la variable de context « dateVeille » et dans value le calcul de la date de la veille à partir de la date du jour. Envoyer le flux de sortie du tMap dans un composant tContextLoad.

Créer un second job contextAffichage, cliquer sur l'onglet Contexts et créer une variable dateVeille de type String. A l'aide d'un tJava afficher dans la console le contenu de la variable de contexte dateVeille. Sauvegarder le job.

Revenir dans le job contextLanceur et faites glisser le nouveau job contextAffichage qui devra s'exécuter à la fin du chargement de la variable de context et cocher la case « Transmettre tout le contexte ».

Exécuter le job contextLanceur et vérifier que la date de la veille s'écrit bien dans la console.



Jobs attendus